



PROGRAMAÇÃO



Escola Naval



António Gonçalves

Agenda — Seleção I



Comparações

Valores lógicos e operadores relacionais.



Decisões simples

Condição, if e indentação.



Duas alternativas

Escolher com if/else.



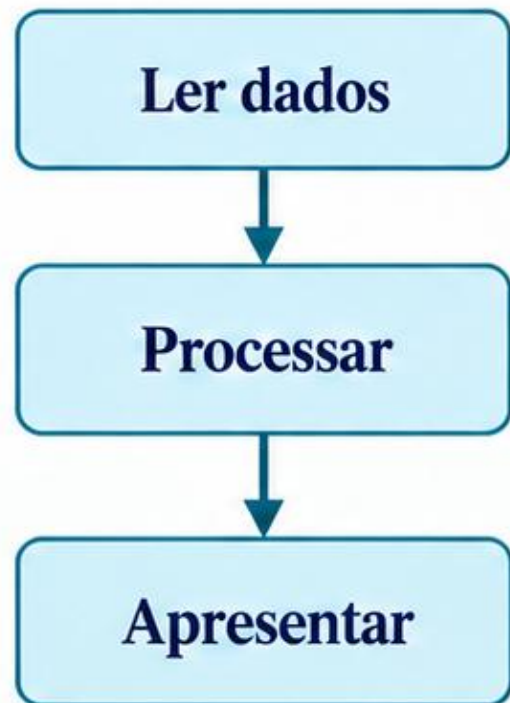
Aplicação e teste

Rastrear a execução e testar os limites.

Execução sequencial e decisão

Execução sequencial

As instruções seguem sempre a mesma sequência.

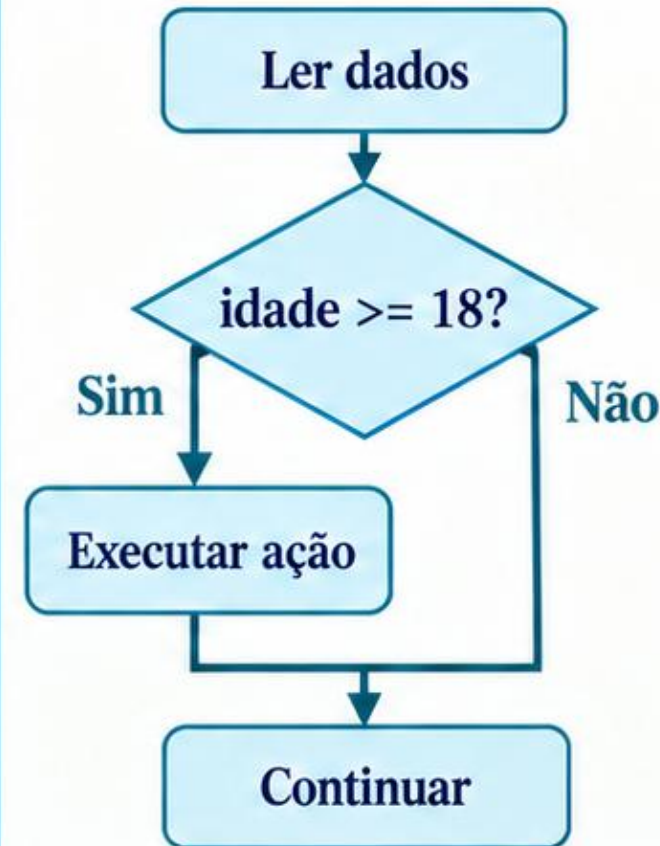


Exemplo:

```
nome = input("Nome: ")
idade = int(input("Idade: "))
print(nome)
print(idade)
```

Decisão

Uma condição permite escolher o percurso a seguir.



Exemplo 1: temperatura

```
temperatura = float(input("°C: "))
if temperatura < 5:
    print("Alerta de frio!")
```

Exemplo 2: nota

```
nota = float(input("Nota: "))
if nota >= 10:
    print("Aprovado!")
```



Ideia principal:

Uma condição permite adaptar o percurso aos dados.

Condições e valores booleanos

Uma comparação produz True ou False.

Os operadores de comparação avaliam uma condição e devolvem um valor booleano: **True** ou **False**.

Tipo booleano em Python

```
type(True)
<class 'bool'>

type(False)
<class 'bool'>
```

Exemplo 1

```
idade = 20
idade >= 18
```

Valores
idade = 20

Comparação
idade >= 18

Resultado
True

Exemplo 2

```
nota = 14
nota < 10
```

Valores
nota = 14

Comparação
nota < 10

Resultado
False



Comparar valores não altera as variáveis.

A comparação apenas avalia a condição. O valor das variáveis mantém-se.



Na memória

Após a comparação, a variável mantém o seu valor original.

```
idade = 20
```

(mantém-se 20)

Operadores relacionais

Operador	Leitura	Exemplo	Resultado
>	maior que	10 > 5	True
<	menor que	10 < 5	False
>=	maior ou igual a	10 >= 10	True
<=	menor ou igual a	10 <= 10	True
==	igual a	10 == 5	False
!=	diferente de	10 != 5	True



O operador define a relação que será testada.

Comparações e valores-limite

Maior que (>)

A condição **idade > 18** é verdadeira apenas para valores estritamente maiores do que 18.



Maior ou igual a (>=)

A condição **idade >= 18** é verdadeira para valores iguais a 18 e para todos os maiores.



Exemplos em tabela

idade	> 18	>= 18
17	False	False
18	False	True
19	True	True

Outros exemplos

```
print(5 < 5)
```



```
print(5 <= 5)
```



> e < excluem o limite.
>= e <= incluem o limite.

Atribuição e comparação

= atribui

`idade = 18`



Memória:

idade

18



Guarda um valor na variável.

== compara

`idade == 18`



True

`idade != 18`



False



Verifica se os valores são iguais.



!= significa “diferente”.



Numa condição de igualdade, utiliza-se **==**.

Da linguagem natural à condição

Linguagem natural	Condição em Python
idade superior a 21	<code>idade > 21</code>
nota inferior a 10	<code>nota < 10</code>
saldo igual a zero	<code>saldo == 0</code>
temperatura diferente de 20	<code>temperatura != 20</code>
valor menor ou igual a 100	<code>valor <= 100</code>



Objetivo

Escolher o operador exige traduzir exatamente o enunciado.

Exercício

Qual é a condição para uma idade de pelo menos 18 anos?

`idade >= 18`

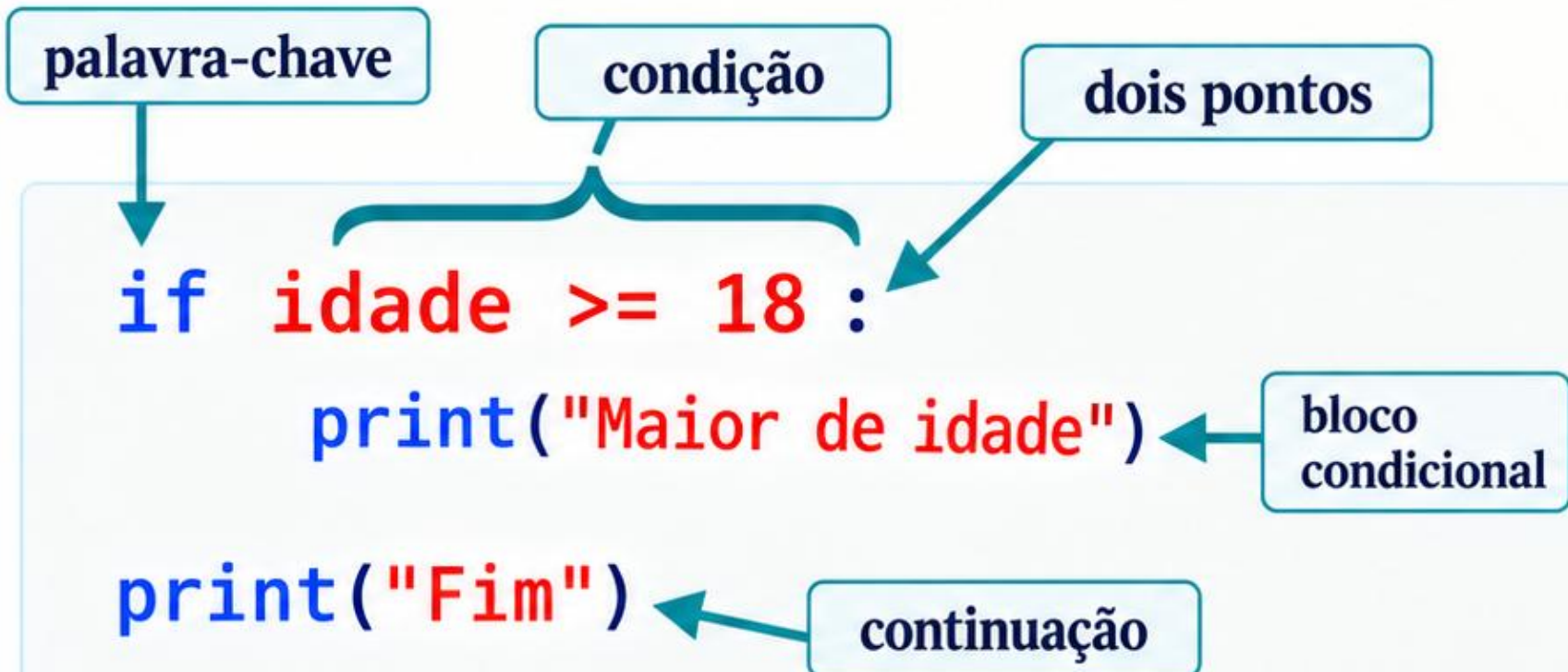


Conclusão

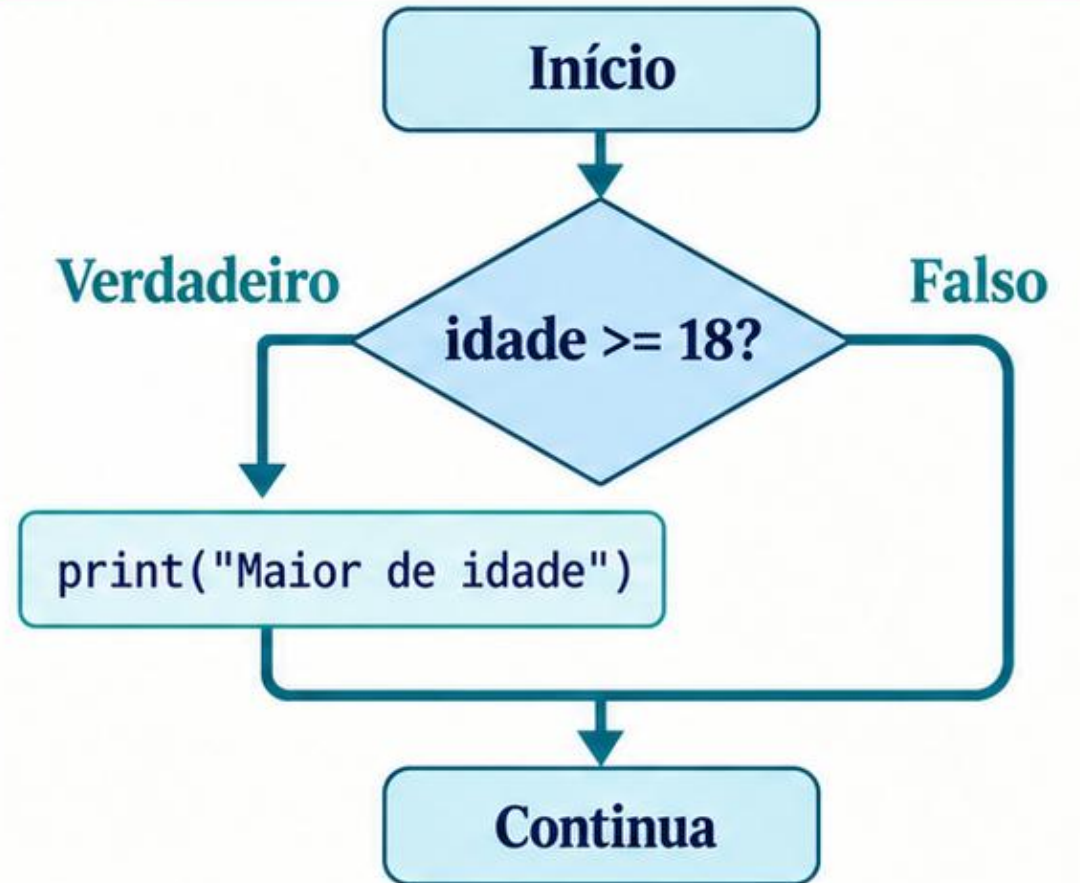
Uma condição em Python é a tradução lógica de uma frase em linguagem natural.

Estrutura if

Código Python



Fluxo de execução



Conclusão:

O bloco só é executado quando a condição é verdadeira.

Execução com True e False

Código Python

```
if idade >= 18:  
    print("Pode entrar")  
print("Fim")
```

Quando a condição é True

idade = 20

20 >= 18 → True

O bloco é executado.

Saída no ecrã



Quando a condição é False

idade = 15

15 >= 18 → False

O bloco é saltado
(não é executado).

O programa continua
depois do if.

Saída no ecrã



Uma condição falsa faz saltar o bloco.
O programa continua depois do if.

Blocos e indentação

Código Python

```
if idade >= 18:
```

```
    print("Maior de idade")  
    print("Acesso permitido")
```

Dentro
do bloco

```
print("Fim")
```

Fora do bloco

Saída no ecrã

Com idade = 20

Maior de idade
Acesso permitido
Fim

Com idade = 17

Fim



A indentação define as instruções associadas à condição.



Cada nível de indentação usa quatro espaços.



4 espaços

Ler e converter antes de comparar

1 Ler do utilizador

A função `input()` lê um valor e devolve um texto (str).

```
valor = input("Idade: ")
```



Valor lido (tipo str):

"20"

2 Converter para inteiro

Use `int()` para converter o texto num número inteiro (int).

```
idade = int(input("Idade: "))
```



Valor convertido (tipo int):

20

3 Comparar

Agora pode comparar o número com 18.

```
idade = int(input("Idade: "))  
if idade >= 18:  
    print("Maior de idade")
```



Resultado da comparação:

True



Atenção!

Comparar diretamente o texto "20" com o inteiro 18 usando `>=` provoca um erro de tipo.

```
valor = input("Idade: ")  
if valor >= 18:  
    print("Maior de idade")
```

TypeError

'>=' not supported between instances of 'str' and 'int'



Conclusão

`input()` devolve texto. Converta dados numéricos antes da comparação.

Aplicação: alerta de temperatura

Problema

Ler a temperatura e apresentar um alerta quando for inferior a 5 °C.

Código Python

```
temperatura = float(input("Temperatura: "))  
if temperatura < 5:  
    print("Alerta de frio")
```

Exemplo 1 – Temperatura baixa



Temperatura: 3
Alerta de frio

Exemplo 2 – Temperatura superior



Temperatura: 12



Nota importante:

Inferior a 5 corresponde a < 5 .

Testar e alterar o valor-limite

Inferior a 5 °C

O teste é verdadeiro para temperaturas menores do que 5 °C.



Código Python

```
if temperatura < 5:  
    print("Alerta de frio")
```

Igual ou inferior a 5 °C

O teste é verdadeiro para temperaturas menores do que ou iguais a 5 °C.



Código Python

```
if temperatura <= 5:  
    print("Alerta de frio")
```

Resultados dos testes

Temperatura (°C)	< 5	<= 5
4	True	True
5	False	True
6	False	False



A mudança de requisito altera o operador.
O teste com 5 distingue as duas regras.

Duas alternativas com if e else

Problema e regra

Pretende-se decidir se uma pessoa pode entrar num determinado local.

A regra é:

- Se a idade for igual ou superior a 18, a entrada é permitida.
- Caso contrário, a entrada não é permitida.

Código Python

```
idade = int(input('Idade: '))  
if idade >= 18:  
    print('Entrada permitida')  
else:  
    print('Entrada não permitida')  
print('Fim')
```

Fluxo de decisão



Ideia principal:

Apenas um dos blocos (do if ou do else) é executado.

O else tem dois pontos, mas não tem uma condição. O if e o else estão ao mesmo nível.

Rastreio de um if/else

Código em Python

O programa usa uma estrutura if/else para definir uma mensagem de acordo com a idade.

```
idade = 17
if idade >= 18:
    mensagem = 'Adulto'
else:
    mensagem = 'Menor'
print(mensagem)
```

Rastreio com idade = 17

A condição é falsa, por isso é executado o ramo else.

Passo	Estado / Condição	Valor
1	idade	17
2	idade >= 18	False
3	Ramo escolhido	else
4	mensagem	'Menor'
5	Saída (print)	Menor

Rastreio com idade = 18

A condição é verdadeira, por isso é executado o ramo if.

Passo	Estado / Condição	Valor
1	idade	18
2	idade >= 18	True
3	Ramo escolhido	if
4	mensagem	'Adulto'
5	Saída (print)	Adulto



O print(mensagem) é executado sempre, independentemente do ramo seguido. A atribuição em cada ramo garante que a variável mensagem fica definida.

Aplicação: par ou ímpar

Ideia principal

Um número inteiro é par quando dividido por 2 o resto é zero. Caso contrário, é ímpar.

Divisão por 2



Código Python

O programa lê um número inteiro, calcula o resto da divisão por 2 e indica se é par ou ímpar.

```
numero = int(input('Número inteiro: '))
if numero % 2 == 0:
    print('Par')
else:
    print('Ímpar')
```

Exemplos de resultados

O resto da divisão por 2 permite determinar se um número é par ou ímpar.

Número inteiro	Resto da divisão por 2	Resultado
-4	0	Par
0	0	Par
7	1	Ímpar
12	0	Par



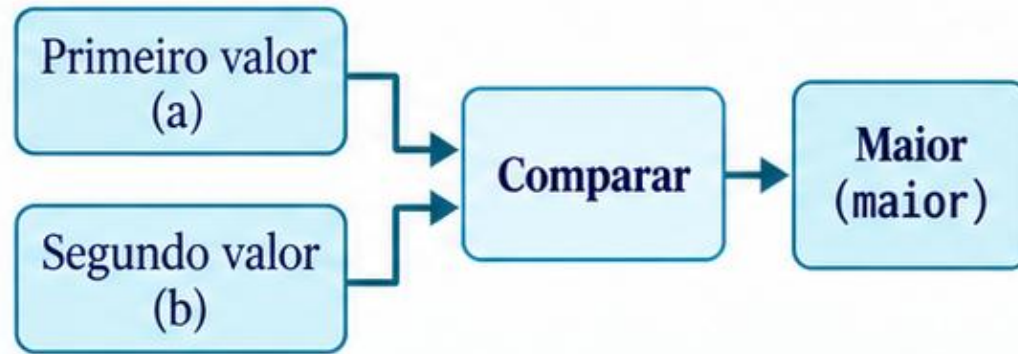
Nota importante:

O operador % devolve o resto da divisão por 2. Um resto igual a 0 significa que o número é par. Assume-se que é inserido um número inteiro válido.

Aplicação: maior de dois valores

Problema

Ler dois valores numéricos e determinar qual é o maior. Em caso de igualdade, é escolhido o primeiro valor (lado da condição).



Não é necessário usar elif, pois ambos os ramos atribuem um valor a maior.

Código Python

```
a = float(input('Primeiro valor: '))
b = float(input('Segundo valor: '))

if a >= b:
    maior = a
else:
    maior = b

print('Maior:', maior)
```

Resultados de testes

a	b	Resultado (maior)
8	3	8.0
2	9	9.0
5	5	5.0

Quando os valores são iguais, a condição é verdadeira e o primeiro valor (a) é escolhido.

O resultado é sempre o maior valor numérico.



Ideia principal:

Uma condição permite seleccionar o maior de dois valores.

Erros de sintaxe

Falta de dois pontos

O cabeçalho do if termina com dois pontos.

✗ Incorreto:

```
if idade >= 18  
    print("Adulto")
```

✓ Correto:

```
if idade >= 18:  
    print("Adulto")
```

Falta de indentação

O bloco de código após os dois pontos deve estar indentado.

✗ Incorreto:

```
if idade >= 18:  
print("Adulto")
```

✓ Correto:

```
if idade >= 18:  
    print("Adulto")
```

= em vez de ==

Para comparar valores, usa-se ==. O operador = serve para atribuir valores a variáveis.

✗ Incorreto:

```
if idade = 18:  
    print("Tem 18 anos")
```

✓ Correto:

```
if idade == 18:  
    print("Tem 18 anos")
```



Estes erros impedem a execução.

Erros de lógica

Regra

Aprovação com nota igual ou superior a 10.



Erro de lógica

Código executa, mas não cumpre a regra.

Código Python (incorrecto)

```
nota = 10
if nota > 10:
    print("Aprovado")
```

Teste com nota = 10

10 > 10 → False

Saída no ecrã:
(nenhuma saída)



Código correcto

Cumpe a regra.

Código Python (correcto)

```
nota = 10
if nota >= 10:
    print("Aprovado")
```

Teste com nota = 10

10 >= 10 → True

Saída no ecrã:
Aprovado



Testar o valor-limite ajuda a detetar o erro.

If independentes ou if/else

If independentes

As condições são avaliadas de forma independente. Ambas podem ser verdadeiras e, por isso, ambos os blocos podem ser executados.

Código Python

```
numero = 150
if numero > 0:
    print('Positivo')
if numero > 100:
    print('Superior a 100')
```

Resultado (numero = 150)

Positivo
Superior a 100

If/else

A instrução if/else escolhe exatamente um dos ramos. Executa o bloco if se a condição for verdadeira e o bloco else se for falsa.

Código Python

```
numero = 150
if numero > 100:
    print('Superior a 100')
else:
    print('Até 100')
```

Resultado (numero = 150)

Superior a 100



Ideia principal:

- Os blocos de dois if independentes podem ambos ser executados.
- A instrução if/else executa exatamente um dos ramos.
- Esta forma de classificação é válida para qualquer valor numérico.
- Não é utilizado elif neste exemplo.

Exercício: preço de entrada

Regra do exercício

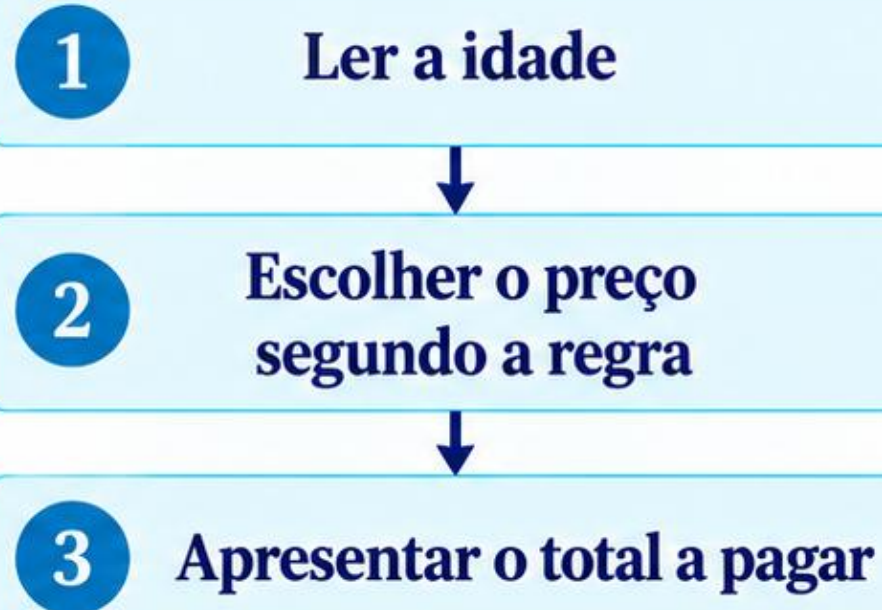
O preço base de entrada é 12 euros.
Os menores de 18 anos têm um desconto de 25%.

Com 18 anos ou mais paga-se o preço normal.

Assume-se que a idade é um número inteiro não negativo.
A validação dos dados será abordada na próxima aula.

O que fazer

Ler a idade, decidir o preço de entrada e apresentar o valor total a pagar.



Resultados esperados

Idade (anos)	Preço de entrada (euros)
17	9.00
18	12.00
19	12.00
0	9.00



Questões para pensar

1. Qual é a condição que permite saber se deve ser aplicado o desconto?
2. Quais são os dois ramos (caminhos) da decisão?
3. Por que razão a idade 18 não tem desconto?

Solução: preço de entrada

Código Python

O programa lê a idade, aplica um desconto de 25% para menores de 18 anos e apresenta o preço a pagar.

```
idade = int(input('Idade: '))
preco = 12.0
if idade < 18:
    total = preco * 0.75
else:
    total = preco
print(f'Total: {total:.2f} euros')
```

Como funciona



Desconto de 25% significa pagar 75% do preço.
A formatação {total:.2f} apresenta o valor com 2 casas decimais.

Execução passo a passo

Traços de execução para duas idades:

Idade = 17

```
idade = 17
preco = 12.0
17 < 18 -> True
total = 12.0 * 0.75
total = 9.0
```

Saída:

Total: 9.00 euros

Idade = 18

```
idade = 18
preco = 12.0
18 < 18 -> False
total = 12.0
```

Saída:

Total: 12.00 euros



Ideia principal:

Uma condição permite aplicar um desconto a menores de 18 anos.

Síntese e verificação

Checklist

- ✓ Comparar com == != < <= > >=
- ✓ Dois pontos após if e else
- ✓ Indentar cada bloco
- ✓ if sem else pode não executar bloco
- ✓ if/else escolhe uma alternativa
- ✓ Converter input antes de comparar números
- ✓ Testar abaixo, no limite, acima

Miniquestão

Qual é a saída do seguinte código?

```
x = 10
if x > 10:
    print('A')
else:
    print('B')
print('C')
```

Resultado

O código produz a seguinte saída:

B
C

A condição é falsa ($10 > 10$ é falso), por isso executa-se o bloco else e, de seguida, o print final.



Próxima aula

Na próxima aula, iremos ver elif, lógica e aninhamento de condições.